

Reopen and resolve

An answer to the Luminal compiler challenge:
exact, explainable search over scheduling and memory decisions

Alberto Hernández-Espinosa

Technical report, September 2026

Abstract

The Luminal challenge asks for a compiler backend that schedules a straight-line program on a small VLIW machine and places its values in a shared scratchpad, so that the program runs in as few cycles and as few scratch words as possible. We answer it with two compilers. The first is a strong *classical* solution, critical-path list scheduling followed by first-fit allocation, which we treat as our benchmark. The second is the compiler we propose, which takes a causal view of the problem: it asks which few decisions are responsible for the current cost, reopens exactly those decisions, and solves them again exactly, keeping a checkable reason for every option it rules out. On the eight public programs the classical compiler scores 1.901379 and ours 2.187957, a gain of 15.1%, against the serial baseline’s score of one. Ours is better on 7 programs and equal on 1, and it is correct on every supplied case. The gain comes from memory: it reduces scratch by 3.316 times against the classical 2.420 times, while giving up a little in cycles (1.444 against 1.494 times). The price is compile time: our compiler spends its budget of 0.1 s, where the classical one needs well under a millisecond; both are far inside the grader’s allowance of 20 s.

The answer in four lines. (1) *Baseline*: the serial compiler, score one by definition. (2) *Classical benchmark*: critical-path list scheduling and aligned first-fit allocation, score 1.901379. (3) *Our proposal*: build a correct first answer from exact queries, then repeatedly reopen the decisions that cause the cost and re-solve them exactly, score 2.187957. (4) *Trade-off*: 15.1% higher score and 24.5% lower geometric-mean $C \times S$ than classical, paid for with roughly 0.1 s of compilation per program.

1 The problem in one page

Luminal builds compilers for AI inference, where performance depends on how tightly work is packed in time and how little memory it occupies while it waits [2]. The take-home challenge [1] is a small, exact version of that problem. A program is a list of operations in dependency order: loads, scalar and vector arithmetic, selects and stores. The machine issues one *bundle* per cycle, and each bundle may contain up to two load, two scalar and two vector operations, one store and one flow operation. An operation’s result can be read only after its *latency*: one cycle for an add, three for a scalar load, four for a vector load. Every result lives in a scratchpad of 256 words of 32 bits; vectors take eight consecutive words starting at a multiple of eight. Two values may share words only if their lifetimes do not overlap.

The compiler must decide two things for every operation: *when* it issues (its cycle) and *where* its result lives (its scratch address). Two numbers measure the result: the number of cycles C , and the footprint S , the highest scratch word used. The score compares both with a frozen serial compiler,

$$\text{score} = \sqrt{\underbrace{\text{geomean}_p \frac{C_p^{\text{serial}}}{C_p}}_{\text{cycle speedup}} \times \underbrace{\text{geomean}_p \frac{S_p^{\text{serial}}}{S_p}}_{\text{scratch reduction}}}, \quad (1)$$

so time and memory weigh exactly the same. For a single program, improving the score is the same as reducing the product

$$J = C \times S. \quad (2)$$

This observation shapes our whole approach: a cycle saved and a proportional amount of scratch saved are worth the same, so a good compiler should be willing to trade one for the other whenever the product falls.

The two decisions pull against each other. Packing a schedule tightly makes many values alive at once, which raises S ; packing memory tightly forces values to wait for each other, which raises C . This coupling is what makes the problem interesting, and it is where a greedy compiler leaves the most on the table.

2 Three compilers, three ideas

The baseline: serial. The challenge supplies a serial compiler that issues one operation per bundle and gives every value a fresh address. It is always correct, never good, and it defines a score of one. On the public programs the supplied starter compiler produces exactly the same schedules and footprints.

The classical benchmark. Our classical compiler is what an experienced compiler engineer would write first, and what the challenge’s own hints point towards. It schedules with a *critical-path list scheduler* [3, 4]: at each cycle it issues the ready operations that head the longest remaining chain of latencies, filling every free slot. It then computes each value’s live interval and places values by *aligned first fit* [5]: each value takes the lowest address that is free for its whole lifetime, vectors on multiples of eight. It tries three fixed orders of the values and keeps the best. This compiler is fast, simple and strong; it scores 1.901379. Its limitation is structural: it decides once, in order, and never revisits a decision. It also optimises time first and memory second, even though the score weighs them equally.

Our proposal: reopen and resolve. The compiler we propose starts from a different question. Instead of asking “what is the best next step?”, it asks “which decisions are responsible for the cost I have now, and is there any way to set them differently that lowers J ?” It answers that question exactly, for small groups of decisions at a time, and repeats. We call this *reopen and resolve*; internally the final version is named C1.

The idea comes from causal deconvolution in algorithmic information dynamics [9, 10, 11]: an observed outcome is explained by separating it into the contributions of the mechanisms that produced it, and each mechanism is studied by intervening on it while the rest is held fixed. Here the outcome is a compiled program, its cost is $C \times S$, and the mechanisms are groups of decisions. The last operations to issue produce C ; the values alive at the memory peak produce S . Reopening one such group while freezing everything else is an intervention, and solving it exactly tells us whether that group can do better. The compiler does not run a general deconvolution algorithm; it borrows this causal stance and applies it to scheduling.

One ingredient makes this practical: the set of legal choices for a group of decisions is kept as an exact and compact description, which can be asked questions such as “what is the smallest legal cycle?” or “is any assignment with J below the current value legal?” without listing its members one by one. How that description is built is not needed to follow this report; what matters is that every answer is exact and every rejected option has a stated cause.

3 Our approach, step by step

Step 1: a correct first answer. Operations are taken in order. For each, the compiler describes the cycles it could use, from the moment its inputs are ready up to a safe horizon, removes the cycles in which its engine is already full, and takes the earliest survivor. Addresses are chosen the same way: the aligned addresses minus those held by values whose lifetimes overlap, and the lowest survivor. Each choice is the answer to an exact question about a set, so it is legal by construction. This first answer is decent but never revisits anything.

Step 2: find the decisions that cause the cost. The compiler builds a *catalogue* of groups of related operations, called windows, each allowed to move its issue cycles within a radius while everything outside the window stays fixed. The catalogue deliberately targets the causes of cost: the k latest operations, which determine C ; the k operations around the memory peak, which determine S ; contiguous blocks; small groups of four; and the whole program when it is small. This is large neighbourhood search [6]: move several related decisions together, because useful improvements rarely come one decision at a time.

Step 3: say exactly what “better” must look like. With the rest of the program fixed, a window’s answer cannot use fewer than L_C cycles or fewer than L_S words. If the current cost is J_0 , a strict improvement must satisfy $C \times S \leq J_0 - 1$, and therefore

$$C \leq \left\lfloor \frac{J_0 - 1}{L_S} \right\rfloor, \quad S \leq \left\lfloor \frac{J_0 - 1}{L_C} \right\rfloor. \quad (3)$$

Every improvement lies inside this box, so the search is confined to it without losing any. If a bound falls below its minimum, the window provably holds no improvement and is answered without searching. This is how the objective of Equation (2) enters the search directly: the compiler never asks for fewer cycles or fewer words in isolation, only for a smaller product.

Step 4: delete impossible options, each with a reason. Before searching, the compiler removes choices that cannot be part of any legal answer. If v reads the result of u and u has latency ℓ , then v cannot issue before u ’s earliest cycle plus ℓ , and u cannot issue after v ’s latest cycle minus ℓ . If an engine’s slots in a cycle are all taken, that cycle is removed from every other operation on that engine. Every removal records a short *certificate* that anyone holding the program can check in one line, in the spirit of explained propagation in constraint solvers [8]. These are the causes in the causal reading: not only “this option is gone”, but “this option is gone because v reads u with latency ℓ ”.

Step 5: search exactly, keep the first validated improvement, repeat. What remains is searched exactly, depth first, abandoning any branch whose best possible J is no better than the current one. A controller keeps 8 windows open at once and gives each a short turn of at most 2 ms, so no single hard window can consume the budget. The first window to find a strict improvement wins: its answer is checked by the challenge’s own validator, becomes the new current answer, and a new round starts from it. The submitted compiler stops after 0.1 s or when a whole round finds nothing. This is the combinatorial view of scheduling and allocation of Castañeda Lozano and colleagues [7], applied to small, well-chosen neighbourhoods rather than to the whole program at once.

An engineering refinement. A search node differs from its parent by a single decision, yet a naive implementation re-checks every constraint at every node. The final version keeps, at each node, the facts its parent had already derived and re-examines only the constraints that the new

decision can reach. Measured on 200 fresh programs at identical search work (10,000 nodes), it makes exactly the same decisions as the version without the refinement in 1,000 of 1,000 paired runs, and its compile call takes 0.574 of the time (97.5% interval [0.552, 0.598]). Because it is faster, it explores more within the same wall-clock budget; at 0.1 s its geometric-mean J is 0.991 of the unrefined version’s (interval [0.985, 0.996]).

4 A worked example: 05_mixed_broadcast

We now follow one public program from the challenge repository through every compiler. Figure 1 shows it: 8 operations, three loads, two `splats` that copy a scalar into all eight lanes of a vector, a vector multiply, a vector add and a vector store. Its difficulty is easy to see. The vector load is slow, several eight-word vectors are alive together, and the order in which the vectors are created decides how much scratch they need.

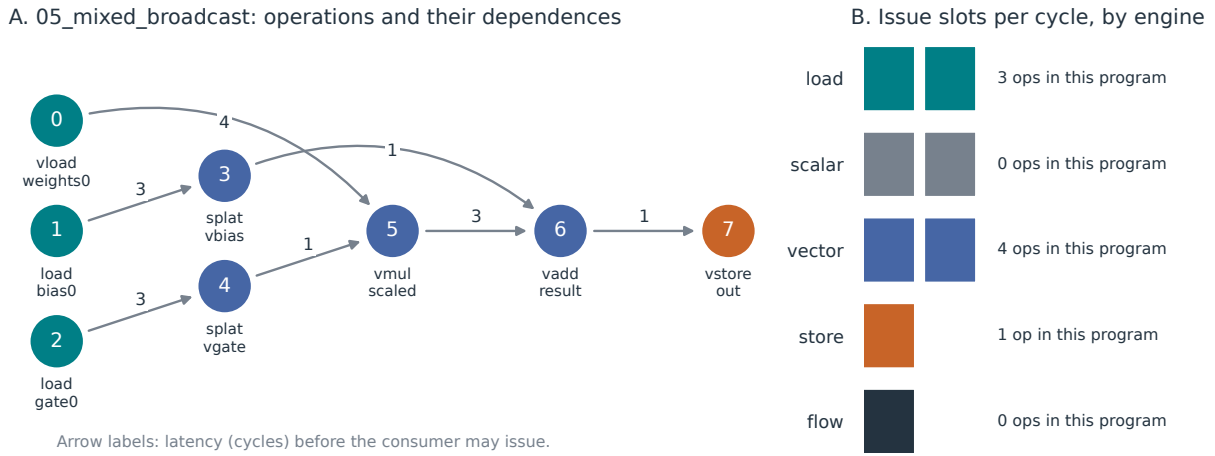


Figure 1: The worked example. (A) Operations and their dependences; each arrow is labelled with the latency that must pass before the consumer may issue. (B) Issue slots per cycle for each engine.

Serial. One operation per cycle and a fresh address for everything: $C = 11$, $S = 42$, $J = 462$. This is the yardstick.

Classical. The list scheduler issues the three loads as early as the two load slots allow, starts the vector work as soon as inputs arrive, and first fit reuses words as soon as values die: $C = 9$, $S = 17$, $J = 153$. It is a big improvement over serial, and on this program the greedy choices are good.

Our Step 1. The first answer takes, for every operation, the earliest legal cycle; for instance operation 2 could issue anywhere in cycles 0 to 7, cycle 0 is already full on the load engine, and so it takes cycle 1. It then gives each value the lowest legal address. The result is $C = 10$, $S = 24$, $J = 240$: legal, but *worse than classical*. Issuing every vector as early as possible keeps two eight-word vectors alive together, and that costs memory. A compiler that stops here loses to the classical one on this program.

Our Steps 2 to 5. The compiler now reopens. It builds a catalogue of 11 windows (Figure 2B). For the window that releases the latest operations, the fixed part implies at least 9 cycles and 8 words, and the box of Equation (3), with the cycle bound also limited by the search horizon,

becomes $C \leq 17$ and $S \leq 26$. Inside it, the declared combinations of issue cycles number 7,873,200; propagation with 5 certificates reduces them to 972,000 before any search begins. The search then walks the path shown in Figure 2C:

- the first round, through the window of the latest operations, keeps the same number of cycles and cuts the footprint: 10×17 ;
- the second round, through the same window, saves a cycle: 9×17 , level with classical;
- the third round, through a small window of four operations that had nothing to offer at the start, saves one more word: 9×16 .

The final answer, $C = 9$, $S = 16$, $J = 144$, beats the classical compiler. It took 4 rounds and 5,476 search nodes, and every intermediate answer was validated by the challenge’s checker. Notice the pattern: a large window opened the way and a small one finished it. Neither kind of move alone reaches this answer.

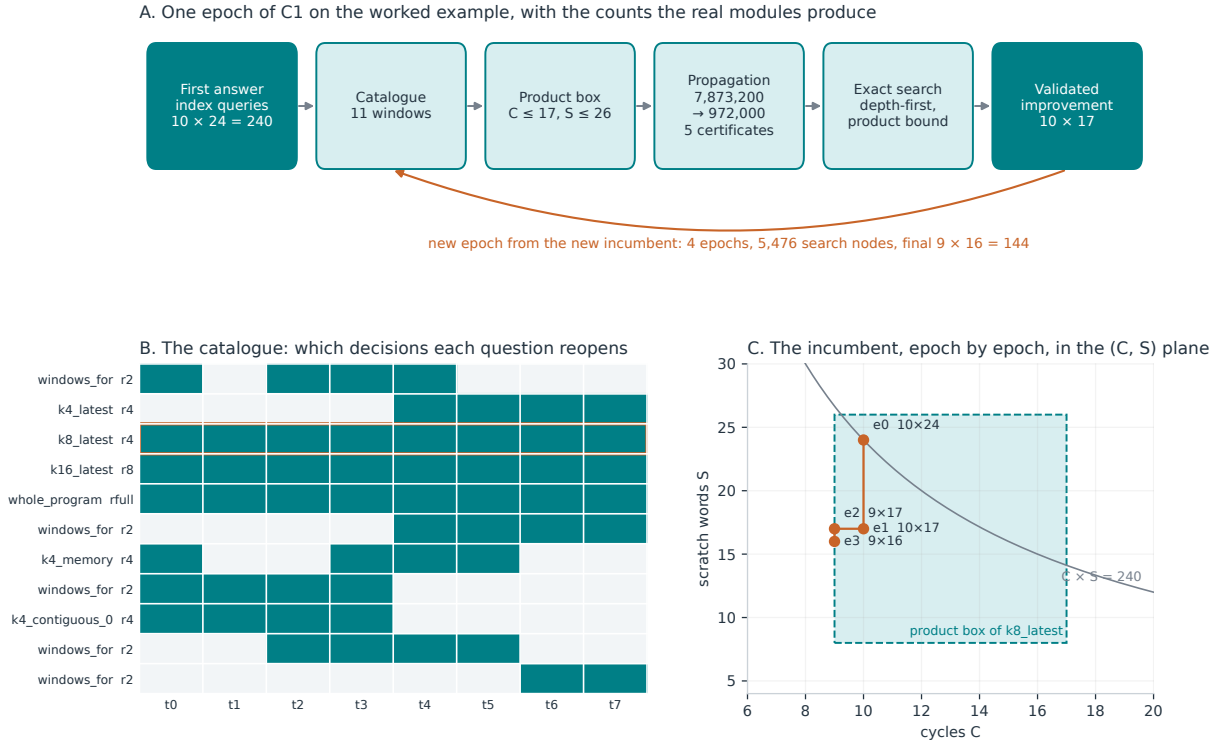


Figure 2: Reopen and resolve on the worked example. (A) One round, with the counts that the real compiler produces. (B) The catalogue: each row is one window and each filled cell an operation it may move. (C) The current answer, round by round, in the (C, S) plane; the curve is the starting value of J , the shaded region the box of Equation (3).

Figure 3 puts the schedules and memory layouts side by side. The difference between classical and ours is visible in the lower panels. Both keep the vectors inside two eight-word blocks; the difference lies in the scalars. The classical layout places one scalar above the vector blocks, which raises the footprint by one word. Ours issues the scalar loads so that each scalar lives in words that no vector occupies at that moment, and the footprint closes exactly at the top of the second block.

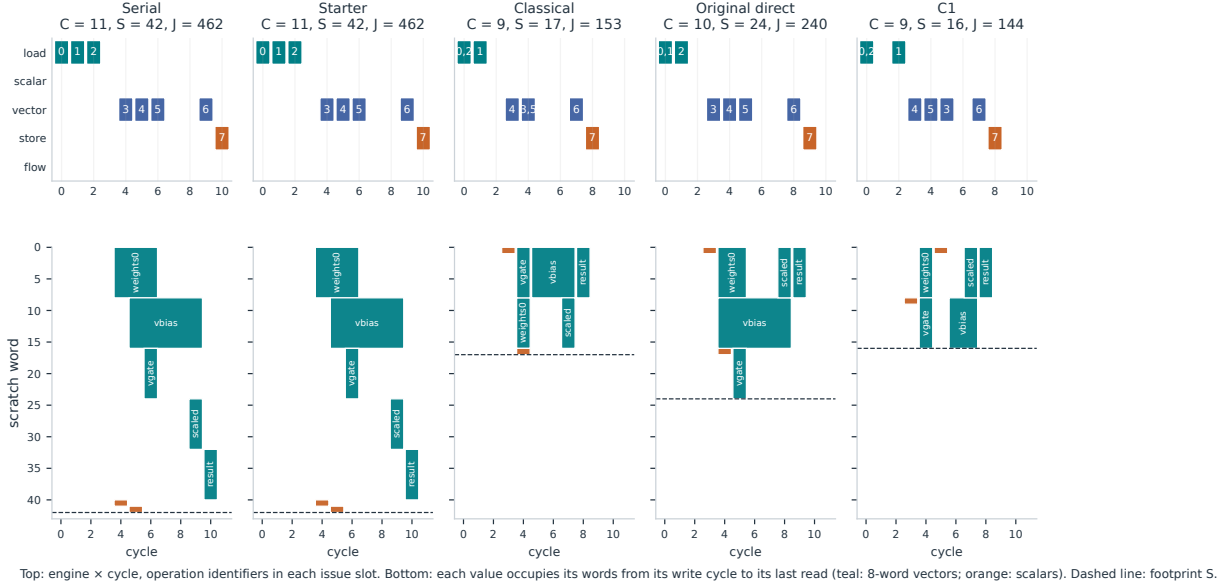


Figure 3: Every compiler on the same program. Top: engine by cycle, with the operations issued in each slot. Bottom: scratch words by cycle; each value occupies its words from its write to its last read, and the dashed line is the footprint S . “Original direct” is our Step 1 alone; “C1” is the full compiler. Every layout was produced by running the compiler and checked by the challenge’s validator.

5 Results against the classical benchmark

5.1 The public programs

Table 1 gives every public program. Our compiler is better than classical in J on 7 programs and equal on 1; it is never worse. Its geometric-mean J is 24.5% below classical, and its score is 2.187957 against 1.901379.

Program	$C \times S$			J		
	Serial	Classical	Ours	Serial	Classical	Ours
scalar_pipeline	18 $\times$ 15	14 $\times$ 6	14 $\times$ 6	270	84	84
scalar_dual_chain	12 $\times$ 9	9 $\times$ 5	8 $\times$ 3	108	45	24
vector_axpy	14 $\times$ 73	10 $\times$ 32	11 $\times$ 16	1,022	320	176
vector_bitmix	16 $\times$ 89	10 $\times$ 40	13 $\times$ 24	1,424	400	312
mixed_broadcast	11 $\times$ 42	9 $\times$ 17	9 $\times$ 16	462	153	144
parallel_memory	21 $\times$ 128	14 $\times$ 56	13 $\times$ 40	2,688	784	520
scalar_selects	20 $\times$ 16	9 $\times$ 6	10 $\times$ 5	320	54	50
vector_reduction	19 $\times$ 137	12 $\times$ 40	12 $\times$ 32	2,603	480	384
Cycle speedup over serial				1	1.494	1.444
Scratch reduction over serial				1	2.420	3.316
Combined score (Equation 1)				1	1.901379	2.187957

Table 1: The eight public programs. “Ours” is the submitted configuration, with a 0.1 s search budget; bold marks the lowest J in each row. Every value was identical in each of 5 fresh-process repetitions, and every compilation passed the challenge’s validator on all supplied cases.

The split between the two halves of the score is the most instructive part of the table. Our compiler uses fewer scratch words than classical on 7 of the eight programs, but fewer cycles on only 2, and more cycles on 3. On `vector_axpy`, for example, it accepts one more cycle to halve the footprint. That is not an accident; it is the objective at work. Because the score weighs time and memory equally, one extra cycle on a program of about ten cycles is a good trade for half

the memory, and a compiler that searches the product $C \times S$ finds such trades, where one that optimises cycles first cannot see them.

5.2 Programs never seen during development

The public programs were visible while we built the compiler, so they show exact results on eight fixed programs, not an estimate of performance on unseen ones. To test generalisation we generated fresh programs in five families (scalar, vector, mixed, dependency-heavy and memory-aliasing) only after each version was frozen. The direct predecessor of our final compiler, which runs the same catalogue and the same depth-first search, was compared with the classical compiler on 200 such programs with a 0.1s budget: its geometric-mean J was 14.4% lower (a descriptive comparison, without an interval), with 146 programs better, 39 equal and 15 worse. An earlier version with the same catalogue, on another fresh cohort of 200 programs, was 10.58% lower (98.33% interval 8.38% to 12.84%), with 138 better, 42 equal and 20 worse. The final compiler adds only the engineering refinement of Section 3, which makes the same decisions at equal work; it was not itself re-measured against classical on a fresh cohort, and we do not multiply results across rounds.

5.3 What it costs

Compile time is the price. The classical compiler compiles a public program in a median of 0.269ms. Ours is designed to use its budget: its median compile call is 100.7ms, and the slowest of its 40 public runs took 0.157s as a whole process, including interpreter start. On the fresh cohort of round one, the search took 76.20 times the classical compile time. The grader allows 20s per program, so both are far inside the limit; the difference is that the classical compiler leaves almost all of that allowance unused, while ours converts a small part of it into better output.

6 Why this approach

It optimises what is scored. Every question the search asks is framed in terms of $J = C \times S$, the quantity the score rewards. Trades between time and memory are therefore found, not forbidden.

It explains itself. Every option the compiler rejects carries a certificate that states its cause, and every accepted change is validated by the challenge’s own checker before it is kept. One can ask of any output why a cycle was not used or why a value moved, and receive a concrete answer.

It proves when to stop looking. When a window’s box is empty, or its exact search finishes without an improvement, that window provably has nothing better to offer given the rest of the program. Effort goes to windows that can still pay.

It is anytime and budget-aware. A correct answer exists after Step 1, and every later round only improves it, so the compiler can be stopped at any moment with a valid schedule. A larger budget buys more quality; a smaller one still returns something good. On the public programs even a budget of 0.01s scores 2.135275.

It generalises by construction. Nothing refers to program names, operation identifiers or constants; the catalogue is derived from the structure of each program. For a company that compiles whole models into single kernels, where a scheduler must place work in time and memory

together, a search that can reopen any group of decisions and explain each change seems to us the right foundation to build on.

7 Correctness and reproducibility

The submitted compiler is a single standard-library Python file that exposes `compile_program` and follows the challenge’s command-line contract. It passes the unchanged public test suite. Beyond the public programs it was run on a corpus of 142 programs and 277 cases, including regression, generated and stress programs that fill the 256-word scratchpad, and passed every case. All 1,248 command-line runs of the exported compiler on fresh and public programs were re-validated with the challenge’s own evaluator. Every number in this report is generated from frozen result files by the report’s build script, which re-derives each value from its source and refuses to compile if any digit in the text is typed by hand.

8 Limitations

The public programs were visible during development, and there is no result on the private grader. The comparison with classical on unseen programs was made for the direct predecessor of the final compiler, not for the final build itself. All timings come from one local machine. Under a real clock, a faster search can take a different path and occasionally end at a worse answer: at 0.1s the final version was better than its predecessor in 123 paired runs, equal in 871 and worse in 6, so quality claims are about the distribution, not about every program. Finally, the classical compiler remains several hundred times faster to compile (median 0.269 ms against 100.7 ms); if compile time mattered more than output quality, it would be the better choice.

Tools and assistance

Claude Code assisted with implementation, review and the preparation of this report; every claim was checked by running the compilers and the challenge’s validator.

References

- [1] Luminal. *Compiler take-home*, pinned revision 573b8a85f4bdb8c3d8ba9f180d5f98dac875c902. <https://github.com/luminal-ai/interview>.
- [2] 3Blue1Brown Talent, *Luminal: AI inference compilers*. <https://www.3blue1brown.com/talent/>.
- [3] R. L. Graham, *Bounds on multiprocessing timing anomalies*, SIAM Journal on Applied Mathematics 17(2), 416–429 (1969).
- [4] P. B. Gibbons and S. S. Muchnick, *Efficient instruction scheduling for a pipelined architecture*, Proc. SIGPLAN ’86 Symposium on Compiler Construction, 11–16 (1986).
- [5] M. Poletto and V. Sarkar, *Linear scan register allocation*, ACM TOPLAS 21(5), 895–913 (1999).
- [6] P. Shaw, *Using constraint programming and local search methods to solve vehicle routing problems*, CP98, LNCS 1520, 417–431 (1998).
- [7] R. Castañeda Lozano, M. Carlsson, G. Hjort Blindell and C. Schulte, *Combinatorial register allocation and instruction scheduling*, ACM TOPLAS 41(3), Article 17 (2019).
- [8] O. Ohrimenko, P. J. Stuckey and M. Codish, *Propagation via lazy clause generation*, Constraints 14(3), 357–391 (2009).

- [9] H. Zenil, N. A. Kiani, A. A. Zea and J. Tegnér, *Causal deconvolution by algorithmic generative models*, Nature Machine Intelligence 1, 58–66 (2019).
- [10] H. Zenil, N. A. Kiani, F. Marabita et al., *An algorithmic information calculus for causal discovery and reprogramming systems*, iScience 19, 1160–1172 (2019).
- [11] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, *Estimations of integrated information based on algorithmic complexity and dynamic querying*, arXiv:1904.10393 (2019).